

Implémentation d'un CNN en C et CUDA pour la reconnaissance de chiffres manuscrits

Nathan Gasser

9 juillet 2026

Résumé

Les réseaux de neurones à convolution (CNN) sont très utilisés de nos jours pour la reconnaissance d'images. Nous allons en implémenter un en utilisant les langages C et CUDA. Son but sera de lire un chiffre manuscrit et sera entraîné sur le dataset MNIST, en utilisant une carte graphique NVIDIA.

Table des matières

1	Introduction	3
2	MLP	3
2.1	Perceptron	3
2.2	Fonctions d'activation	4
2.2.1	La fonction ReLU	4
2.2.2	La fonction tanh	4
2.3	Backpropagation	5
2.4	Plusieurs neurones	5
3	CNN	5
3.1	Convolution	5
3.2	Pooling	5
3.3	CNN complet	5
4	Implémentation en C et CUDA	5
4.1	Moteur d'inférence en C	5
4.2	Moteur d'entraînement en CUDA	5
4.3	Interface utilisateur	5

1 Introduction

Pour la reconnaissance de chiffres manuscrits, il y a plusieurs méthodes possibles. Une des méthodes les plus simples est le perceptron à plusieurs couches (MLP), mais les CNN sont plus efficaces pour les tâches de reconnaissance d'images. Un CNN est un type de réseau de neurones qui a pour particularité d'utiliser des opérations de convolution pour extraire des caractéristiques de l'image. Ce TM aura pour but d'implémenter ce type de réseau de neurones en CUDA pour la partie entraînement et par la suite en C pour la partie inférence.

2 MLP

2.1 Perceptron

Le perceptron est la brique de base de l'intelligence artificielle. Il accepte un nombre fixe d'entrées (inputs) (N) et nous donne un seul nombre en sortie (output). Le perceptron peut en lui même déjà être utilisé pour créer des systèmes de classification simples.

Chaque entrée s'appelle x_n (où n est un chiffre entre 0 et N non inclu). Pour chaque entrée, il y a un poids (weight) associé (w_n). En plus des entrées et des poids, il y a un biais (bias) (b). Pour calculer la valeur de sortie, il faut également choisir une fonction d'activation (f). Il peut s'agir théoriquement de n'importe quelle fonction mathématique qui n'accepte qu'une seule entrée, mais dans le cadre de ce TM, nous allons utiliser deux fonctions principalement : ReLU et tanh

Pour calculer la valeur de sortie de notre perceptron, il faudra faire une somme pondérée de nos entrées à laquelle nous allons additionner le biais avant de faire passer cette valeur dans la fonction d'activation :

$$y = f\left(\sum_{n=1}^N w_n x_n + b\right)$$

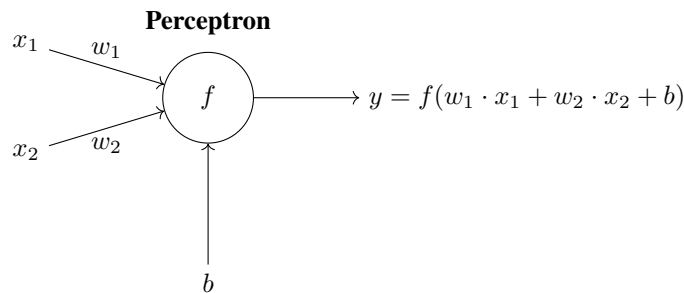


FIGURE 1 – Schéma d'un perceptron avec deux entrées (x_1 et x_2), deux poids (w_1 et w_2), un biais (b) et une fonction d'activation (f) qui produit la sortie (y).

2.2 Fonctions d'activation

La fonction d'activation permet d'ajouter de la non linéarité au modèle. Cela permet au modèle d'apprendre des motifs bien plus complexes. Sans fonction d'activation, le modèle ne pourrait apprendre que des motifs linéaires, ce qui est beaucoup trop limité pour des tâches de reconnaissance d'images.

2.2.1 La fonction $ReLU$

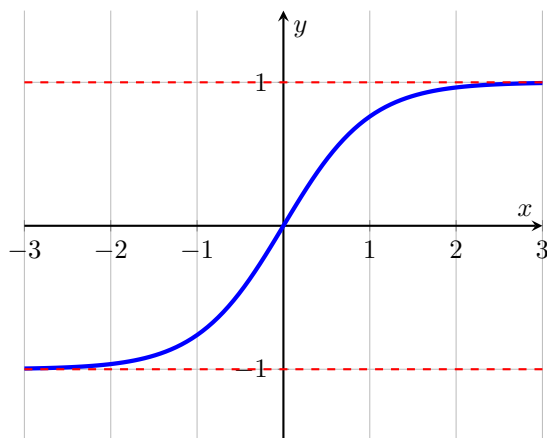
La fonction $ReLU$ est la suivante :

$$ReLU(x) = \begin{cases} 0 & \text{si } x < 0 \\ x & \text{si } x \geq 0 \end{cases}$$

2.2.2 La fonction $\tanh$

La fonction $\tanh$ (tangente hyperbolique) est utilisée pour confiner les valeurs de sorties entre la valeur -1.0 et 1.0 . Dans le contexte de la reconnaissance d'images, elle permet de représenter le niveau de certitude de notre modèle par rapport à différentes prédictions. Les fonctions softmax ou sigmoid sont également souvent utilisées mais par simplicité, j'ai décidé d'utiliser la fonction $\tanh$.

Graphe de $\tanh(x)$



2.3 Backpropagation

2.4 Plusieurs neurones

3 CNN

3.1 Convolution

3.2 Pooling

3.3 CNN complet

4 Implémentation en C et CUDA

4.1 Moteur d'inférence en C

4.2 Moteur d'entraînement en CUDA

4.3 Interface utilisateur